

论文解构报告

SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills

Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee

2023

Pipeline Parallelism

LLM Serving

Chunked Prefill

Delay Scheduling

源文件：<https://arxiv.org/pdf/2308.16369>

arXiv：<http://arxiv.org/abs/2308.16369>

DOI：[10.48550/arxiv.2308.16369](https://doi.org/10.48550/arxiv.2308.16369)

生成时间：2026-07-20

目录

- 摘要翻译 3
 - 1. 一句话总览 3
 - 2. 阅读范围与证据状态 3
 - 3. 根问题：论文究竟要解决什么 3
 - 4. 旧方法为何不够 4
 - 5. 核心洞见与假设 4
 - 6. 方法：从洞见到可执行系统 5
 - 7. 为什么它可能有效 5
 - 8. 实验与指标：证据是否支撑主张 5
 - 9. 图表解读与论证关系 8
 - 10. 完整因果推理树 11
 - 11. 结论、贡献与边界 12
 - 12. 面向初学者的学习路径 12

摘要翻译

由于单张 GPU 的显存容量不足以容纳大语言模型（LLM），模型并行已成为在多张 GPU 上运行 LLM 服务的标准方法。在在线服务环境中，张量并行由于可以通过并行执行来降低计算延迟，已成为单节点多 GPU 系统中事实上的标准方法。尽管流水线并行能够提供更高的吞吐量，但它面临着在在线工作负载下进一步加剧的流水线不平衡问题，从而导致资源利用率低下和性能退化。在本研究中，我们重新审视了用于 LLM 服务的流水线并行。我们的分析表明，在在线服务中，流水线各个阶段之间的计算不平衡会更加恶化。为了解决这些流水线低效问题，我们提出了三种技术：两种通过动态调整分块大小来缓解预填充引起的气泡的机制（贪心和预测方案），以及一种动态重新平衡流水线阶段之间解码工作负载以进一步减少流水线气泡的延迟调度技术。我们在 SGLang 之上实现了我们的技术，并证明在四张 NVIDIA A100 40GB GPU 上运行 Qwen2.5 32B 和 14B 时，结合了我们所提机制的流水线并行性能优于张量并行。

1. 一句话总览

论文元数据

字段	内容
标题	无法从当前 OCR 内容确认
作者	文中未说明
发表场所 / 年份	文中未说明
研究领域	大语言模型服务 / 分布式系统
关键词	文中未说明

资源链接

- 原始文件：本地上传文件
- arXiv：文中未说明
- DOI：文中未说明
- 代码 / 数据集：文中未说明

标签 (4 个)：流水线并行 · 动态分块预填充 · 延迟调度 · LLM 在线服务

[作者明确陈述] 本段旨在总览本研究的主要贡献与性能表现，针对在线大语言模型服务中的流水线并行（PP，纵向流水线装配，将模型的不同层依次分配给不同的 GPU 阶段，像装配线一样顺序处理微批次的工作方式 [页码：4]）气泡问题，论文提出了动态调整分块大小的贪婪与预测机制（DCP，动态分块预填充，根据系统负载和服务等级协定（SLO，服务提供商向用户做出的服务性能承诺，如首字延迟不超过 2 秒 [页码：2]）约束实时增减分块大小的调度机制 [页码：6, 7]）以及重新平衡生成阶段负载的延迟调度技术（DS，允许部分生成任务稍微推迟，通过重组微批次大小对齐硬件高效边界来拉平各流水线阶段的计算耗时 [页码：3, 9]），在低带宽的 PCIe（快速外设部件互连标准，用于连接主板和硬件加速芯片的低带宽通道 [页码：4]）4 卡 GPU（图形处理器，专用于大规模并行计算的硬件加速芯片 [页码：3]）集群上，实现了好吞吐量（Goodput，在完全满足 SLO 约束的前提下系统能承受的最大请求频率 [页码：3]）对张量并行（TP，横向分工计算，将大模型每一层的计算权重切分给多个 GPU 共同计算，完成后需要全局同步通信的工作方式 [页码：4]）和静态 PP 的显著超越 [页码：2, 3]。

2. 阅读范围与证据状态

[基于文中逻辑的推断] 为了明确评估结论的依据与局限性，本段首先分析了本研究的阅读范围与证据完整性状态，本篇报告基于论文第 2 页至第 15 页以及第 203 页的文本 OCR（光学字符识别，将图片中的文字识别并转换为可编辑文本的技术）数据。
[基于文中逻辑的推断] 为了界定当前分析的可靠性边界，本段梳理了文本与图像证据中存在的明确缺失项，目前元数据中的标题、作者、发表场所和关键词因 OCR 缺失而无法确认，且系统的 arXiv（开放学术论文预印本平台）、DOI（数字对象标识符，用于唯一标识数字化文献的编码）以及代码/数据集链接均在文中未说明，同时延迟调度的对齐阈值配置、AMD GPU / NPU（神经网络处理器，专为深度学习算法设计的加速芯片 [页码：4]）的定量验证结果同样在文中未说明，此外有 14 张图表由于系统限制未能进行视觉分析，这构成了当前的证据状态缺口。

3. 根问题：论文究竟要解决什么

[背景知识] 为了理解本论文的分布式计算背景，本段说明了大模型在线服务的硬件存储挑战，当单卡 GPU 显存不足以容纳大语言模型时，必须使用模型并行（MP，由于大语言模型参数量极其庞大，无法被放入单个 GPU 的显存中，因此必须将模型切分成多个部分分布在多张显卡上协同运行的并行策略 [页码：3]）技术将参数切分分布在多张显卡上运行 [页码：2, 3]。
[基于文中逻辑的推断] 为了明确本研究的技术切入点，本段阐明了在低带宽互连硬件下分布式并行的瓶颈问题，张量并行在 PCIe 低带宽下通信极为频繁并带来极高延迟，而流水线并行虽然通信开销极小，但在在线请求波动和输入长度不确定的场景下极易产生严

重的流水线气泡 (Pipeline Bubble, 流水线并行的装配线上, 由于各个处理工位的加工时间不一致, 导致某些 GPU 阶段闲置等待的无用空载时间 [页码: 2, 4]), 导致 GPU 资源利用率低下 [页码: 2, 4]。

[基于文中逻辑的推断] 为了衡量服务的优化成效, 本段定义了 LLM 在线服务所采用的成功标准, 系统必须保障首字延迟 (TTFT, 用户发送请求后, 系统处理完所有输入提示词并输出第一个单词所需要的时间, 对应预填充阶段 [页码: 3]) 和单字生成延迟 (TPOT, 在首字输出后, 系统一个接一个生成后续单词的平均间隔时间, 对应生成阶段 [页码: 3]) 满足服务等级协定, 在此前提下最大化系统的好吞吐量以降低服务运营成本 [页码: 3]。

[基于文中逻辑的推断] 为了揭示本论文的核心创新逻辑, 本段总结了本论文的 story, 针对在仅有 PCIe 互连的 GPU 服务器上张量并行通信开销大、静态 PP 流水线不平衡气泡严重的局限, 本文引入了动态调整分块大小的贪婪与预测机制以及延迟调度技术, 通过最大程度平抑预填充与生成阶段的多类流水线气泡, 成功在低带宽服务器下实现了流水线并行在线服务性能对张量并行的超越 [页码: 2, 3]。

4. 旧方法为何不够

[实验支持] 为了分析已有分布式并行方案的局限性, 本段指出了张量并行在低带宽硬件环境下的通信瓶颈, 在仅有 PCIe 互连、缺乏高带宽 NVLink 互连的服务器上, 张量并行频繁的 all-reduce 同步通信会迅速吃满带宽并产生极高延迟, 限制了多卡加速效果 [页码: 2, 4]。

[作者明确陈述] 为了阐述现有流水线并行框架的问题, 本段说明了静态流水线并行调度的设计缺陷, 由于其将请求固定在分配好的微批次中而无法在运行时调整, 无法应对在线服务中的动态负载不均衡, 从而在各个流水线阶段之间重复引发迭代停顿和气泡 [页码: 3, 6]。

[实验支持] 为了分析输入分块机制的缺陷, 本段指出了静态分块预填充方案所面临的两难困境, 大分块在低负载下会因为少数长预填充主导流水线而引发严重的 P-P (预填充-预填充) 与 P-D (预填充-生成) 不平衡气泡, 而小分块则由于 GPU 硬件算力未被充分利用导致吞吐量大幅受损, 在请求到达率高时排队严重, 恶化 TTFT [页码: 2, 3, 5, 6]。

[实验支持] 为了分析生成阶段的硬件效率限制, 本段解释了线性层计算在非友好边界上的效率损失, 由于硬件 kernel 层面具有显著的阶梯效率 (如 batch size 从 128 微增到 129 时耗时骤增约 31%), 现有的生成调度没有对齐和迁移机制, 导致生成阶段产生大量的 D-D 不平衡气泡 [页码: 5, 6]。

[背景知识] 为了对比与 ORCA 系统的技术差异, 本段梳理了 ORCA (一种提出迭代级调度和选择性批处理以提高 GPU 利用率的早期大模型推理框架 [页码: 15]) 系统与本方法的演进关系, ORCA 原本试图通过引入迭代级调度和选择性批处理来提高单个 GPU 的计算利用率, 但它依赖于单节点且无流水线并行的前提, [作者明确陈述] 本文批判其没有考虑混合预填充与生成以及长提示词时产生的严重干扰, 更忽略了分布式流水线并行环境下的气泡开销。本文用动态分块调整机制回应这一限制, 并通过混合预填充与生成轨迹下的好吞吐量与延迟对比实验检验这一点 [页码: 2, 3]。

[背景知识] 为了对比与 Sarathi-Serve 系统的技术差异, 本段梳理了固定分块方案与动态分块方案的演进关系, Sarathi-Serve (一种引入固定大小分块预填充来平衡首字和生成延迟的大模型推理框架 [页码: 2, 3, 15]) 原本通过固定大小的分块预填充来降低长 prompt 导致的生成延迟突增, 但其依赖于静态分块大小的前提, [作者明确陈述] 本文批判其固定分块大小在消除气泡与 GPU 计算吞吐量之间产生了两难权衡, 在多卡 PP 环境下仍保留大量气泡导致好吞吐量落后于 TP。本文提出动态分块预填充机制 (DCP) 回应这一限制, 并通过图 4 中气泡时间的请求率敏感性实验以及图 9、10 中 DCP 相比静态 PP 的好吞吐量对比实验对其进行了检验 [页码: 3, 6, 10, 11] [实验支持]。

[背景知识] 为了对比与 Niyama 系统的技术差异, 本段梳理了局部自适应分块方案与全局流水线协调调度方案的演进关系, Niyama (一种根据局部节点 SLO 状态动态调节预填充 token 数量的近前沿推理框架 [页码: 15]) 原本通过根据当前 SLO 状态动态调节预填充 token 的处理数量来优化推理, 但其依赖于单节点局部自适应的前提, [作者明确陈述] 本文批判其仅基于单节点进行局部分块而忽略了多卡流水线并行的结构性约束, 因而在 PP 下仍无法消除气泡。本文通过将流水线深度 D_p 纳入决策边界的动态分块算法回应这一限制, 并使用不同 SLO 紧绷度下的 Goodput 敏感性实验 (图 12) 对其进行了检验 [页码: 7, 8, 12, 13] [实验支持]。

[背景知识] 为了对比与物理相位解耦方案的技术差异, 本段梳理了 Splitwise 等物理相位解耦系统与本方法的演进关系, Splitwise, Mooncake, Dist-Serve [页码: 15] (通过物理分离预填充和生成阶段在不同物理 GPU 节点上运行的相位解耦推理服务架构 [页码: 15]) 原本试图通过将预填充和生成阶段在物理上分离在不同的 GPU 节点上运行以消除两阶段间的计算干扰, 但其依赖于高带宽的节点间网络以在物理节点间传输 KV 缓存 (键值缓存, 用于存储大模型推理中历史 Token 状态以避免重复计算的内存区域 [页码: 8]) 的前提, [作者明确陈述] 本文批判其物理分离在跨节点时引入了模型参数双重存储以及搬运庞大 KV 缓存需要极高通信带宽的巨大系统性开销。本文通过 DCP 机制允许在同一 GPU 节点上统一调度两阶段来回这一限制, 文中虽然未进行直接的分布式网络对比实验, 但以自适应延迟预测模型极低的在线运行耗时提供了低开销调度的间接证据 [页码: 14, 15, 203 (表格)] [实验支持]。

5. 核心洞见与假说

[基于文中逻辑的推断] 为了说明本论文关于动态调整分块的设计理念, 本段阐明了动态分块的帕累托平衡洞见, 主张分块大小应根据系统负载和 SLO 约束实时增减, 在低负载下主动缩小分块以规避气泡 (降低 TPOT), 在高负载下主动扩大分块以最大化 GPU 吞吐量 (降低 TTFT), 只要其不超过 TPOT SLO 限制 [页码: 6, 7]。

[作者明确陈述] 为了说明延迟预测的前瞻性设计, 本段阐述了自适应延迟预测 (ALP, 结合离线测量的线性层延迟与在线自适应校准的注意力层延迟实时预测硬件迭代时间的轻量算法 [页码: 8]) 模型引入离线与在线解耦预测的洞见, LLM 迭代延迟由线性层决定部分 (与分块大小呈线性关系, 可离线建模) 与注意力层决定部分 (由递推最小二乘法 (RLS, 根据最近的预测误差自动在线修

正参数的数学算法 [页码：8] 算法根据最近预测误差在线修正参数并自适应校准和学习噪声) 组成，可以精准预测未来迭代延迟，消除贪婪算法的反馈滞后 [页码：7, 8] [实验支持]。

[实验支持] 为了解决生成阶段微批次请求数不均的硬件效率问题，本段阐明了基于延迟调度的 Decode 重平衡洞见，通过在 $TPOT_{slack}$ 充裕或紧张时进行微批次间的生成请求抢占与延迟迁移，强行将微批次大小重组并对齐到 128、256 等硬件高效边界，从而规避线性层计算在非对齐边界上的阶梯效率折损，消除 D-D 不平衡气泡并加速 KV 缓存回收 [页码：3, 9]。

[基于文中逻辑的推断] 为了界定本研究核心方案的可行性基础，本段梳理了该方法所依赖的技术前提与不成立条件，其可行性依赖于大模型生成阶段的线性层计算具有显著的 128 对齐硬件阶梯效率特征，且注意力层的在线参数能通过 RLS 快速收敛；而在 SLO 设为极度严苛（如 $SLO\ scale \leq 0.2$ ）或物理算力绝对不足时，调度算法的优化空间将被完全压缩，导致好吞吐量退化为零，即上述假设前提不再成立 [页码：5, 6, 8, 12]。

6. 方法：从洞见到可执行系统

[基于文中逻辑的推断] 为了呈现系统的整体工作架构，本段总览了从请求输入到结果输出的完整数据流，当在线大语言模型请求到达时，系统接收请求信息并监控当前流水线状态，首先通过 ALP 模型在线估计不同分块大小对应的硬件延迟，据此由 DCP 调度器动态计算出下一步要执行的预填充分块大小 c ，同时由延迟调度模块重组生成阶段微批次的请求数量以对齐硬件友好边界，最终多卡流水线共同执行微批次计算并输出生成的 token，同时利用实测延迟实时更新预测模型参数 [页码：7, 8, 9]。

[基于文中逻辑的推断] 为了解决静态分块大小无法自适应系统负载导致流水线气泡的挑战，本段详细阐述了贪婪动态分块调度 (DCP_greedy) 模块的运行机理，该模块在运行中根据前一次迭代的 $TTFT_{slack}$ 、 $TPOT_{slack}$ 、当前可用 KV 缓存比例 KV_{free} 以及预设常数边界 $\alpha, \beta, \gamma, \theta_{mem}$ ，每隔流水线深度 D_p 轮迭代更新一次分块大小。当 $KV_{free} < \theta_{mem}$ 时直接重置分块大小至安全线以释放内存，否则根据延迟裕量增减分块大小（如 $TPOT_{slack} < 0$ 或 $TTFT_{slack} > \alpha \cdot TTFT_{SLO}$ 时减小分块，TTFT 紧绷时增大分块），DCP_greedy 调度方法的技术优势在于改动极小且易于集成 [页码：7]。

[作者明确陈述] 为了消除贪婪算法的反馈滞后，本段阐述了预测动态分块调度 (DCP_pred) 与 ALP 模块的联合运行流程，ALP 模块首先利用 RLS 算法根据上一轮预测误差在线校准注意力延迟模型的关键参数 $\beta_0, \beta_1, \beta_2$ ，然后评估各候选分块大小 c 下的预期迭代延迟 $T_{iter} = T_{offline} + T_{online}$ ，过滤掉会导致 $T_{iter} \times D_p$ 超出 $TPOT_{SLO}$ 的分块大小，并根据待处理队列剩余时间确定最低预填充吞吐量下限以筛选分块，最后选择可行域内的最小分块以消除气泡（系统过载无可行解时选满足约束的最大分块最大化吞吐），DCP_pred 调度方案的技术优势在于能主动防范尾部延迟剧增 [页码：8, 9] [实验支持]。

[实验支持] 为了消除生成阶段微批次不均及线性层在非友好边界上耗时骤增约 31% 的阶梯效率瓶颈，本段阐述了延迟调度模块的重组与迁移流程，该模块接收各阶段微批次的生成请求数分布，如果微批次请求数略微超出了 128 的倍数（如 128, 256, 384）且在设定的溢出阈值内，则临时抢占该微批次中的多余请求以强行向下对齐至对齐边界；如果超出量过大，则将批次大小对齐至所有阶段的平均请求数进行微批次迁移与重组；在抢占时，当 $TPOT_{slack}$ 紧张时优先抢占旧请求以保新请求延迟，在充裕时优先抢占新请求以加速旧请求结束以腾退 KV 缓存，延迟调度技术的技术优势在于能拉平 D-D 气泡并显著降低硬件算力浪费 [页码：5, 6, 9]。

7. 为什么它可能有效

[基于文中逻辑的推断] 为了论证动态分块预填充 (DCP) 机制的有效性，本段分析了其由机制到收益的逻辑链条，DCP 通过在低负载时缩小分块、在高负载时增大分块的机制，能在低负载下规避长 prefill 带来的流水线气泡以降低 TPOT，在高负载下提升 GPU 的计算吞吐以降低排队延迟和 TTFT，DCP 的生效前提是系统需要能够实时获得相对准确的负载状态与延迟裕量 [页码：6, 7]。

[基于文中逻辑的推断] 为了论证预测调度 (DCP_pred) 的有效性，本段分析了自适应延迟预测模型结合 RLS 算法的逻辑传导，该模型通过在线自适应修正 attention 延迟参数，能够主动评估分块大小对未来数轮流水线迭代延迟的影响，从而做出主动式的调度以克服贪婪算法的反馈滞后并减少尾部延迟，ALP 预测模型的生效前提是 RLS 算法在运行中需具备极高的计算效率，且离线测得的线性层耗时表保持稳定 [页码：7, 8]。

[基于文中逻辑的推断] 为了论证延迟调度 (DS) 的有效性，本段分析了微批次抢占对齐机制的逻辑传导，在微批次超出 128 的倍数在一定阈值内时，通过临时抢占多余请求使批次大小向下对齐到边界，能规避线性层在非对齐边界上的阶梯效率折损，拉平各阶段的计算耗时并消除 D-D 不平衡气泡，DS 的生效前提是被抢占的少量请求所引入的 TPOT 性能损失在 SLO 约束的可承受范围内 [页码：5, 6, 9]。

8. 实验与指标：证据是否支撑主张

[基于文中逻辑的推断] 为了对本研究的实验设计进行分类说明，本段概述了实验的体系结构，本文的实验主要分为三类：第一类是比较实验，在 PyTorch（一种主流的开源深度学习软件框架 [页码：9]）、CUDA（NVIDIA 推出的通用并行计算架构和底层算子运行环境 [页码：9]）及 FlashInfer（专为 LLM 注意力层计算加速的高性能算子库 [页码：9]）环境下，在 Qwen2.5 32B/14B 模型上对比 TP、 PP_{static} 以及本文提出的 DCP/DS，回答该方案是否优于已有并行与调度基线；第二类是消融/敏感性实验，包括 ALP 模型中各注意力延迟项的精度消融，以及 SLO 紧绷度对 Goodput 提升幅度的敏感性分析；第三类是挑战性案例，在输入输出长度完全反转的 CNN-Reversed 轨迹下评估生成重负载时的性能上限 [页码：9, 10, 13, 14]。

[基于文中逻辑的推断] 为了直观展示本论文中提出的方案与被批评的已有方案之间的对应关系，以下以表格形式展示了比较工作与批判性限制的呼应，详细梳理了被批评路线的优势、限制、本文的对应模块、实验检验以及尚未覆盖的适用边界。

比较工作与被批判限制的呼应

比较工作/基线	它试图解决的问题或优势	本文指出的具体 limit/失效条件	本文的对应模块	实际比较实验、指标和结果	仍未验证的边界
张量并行 (TP)	横向切分模型每一层的矩阵乘法权重以加速单步计算 [页码 :4] [背景知识]	在 无 高 带 宽 NVLink 互连、仅有 PCIe 的低带宽单节点上，高频的 all-reduce 同步通信会吃满带宽并产生极高延迟，限制多卡加速 [页码 : 2, 4] [实验支持]	动态分块调整 (DCP) 与延迟调度 (DS) 优化下的流水线并行 (PP) [页码 : 2, 3]	在 4 卡 PCIe A100 上，使用 Qwen2.5-32B 模型运行 AzureConv，静态 PP 的 Goodput 在 SLO 限制下 (3.25 req/s) 略逊于 TP (4.0 req/s)，但在引入 DCP 调度后大幅反超 TP [页码 :3, 11] [实验支持]	跨节点多机分布式网络下或更大 GPU 规模下的通信开销与性能对比 [页码 : 9] [基于文中逻辑的推断]
静态流水线并行 (PP_{static})	纵向拆分模型层以分摊计算，实现极低的通信开销 [页码 : 4] [背景知识]	在线服务中流量随机到达和提示词长度不均，静态调度导致严重的 P-P、P-D 和 D-D 不平衡气泡，降低资源利用率 [页码 : 3, 6] [作者明确陈述]	动态分块调整 (DCP) 与延迟调度 (DS) [页码 :3, 6, 9]	在 AzureConv/ CNN/ShareGPT 轨迹下，DCP_pred 相比 PP_{static} 在 32B 模型上分别降低了 35%/42%/36% 的 TPOT 延迟，并显著提升 Goodput [页码 : 11] [实验支持]	8 卡或更大规模 PP 上的扩展性，以及更大流水线深度时的预测模型累积误差 [页码 :9] [基于文中逻辑的推断]
固定分块预填充 (Static Chunked-Prefill)	固定大小切分长 prompt，缓解首字计算对生成阶段的干扰 [页码 : 2, 3] [背景知识]	静态分块大小无法自适应系统负载。大分块在低负载下引发严重的 P-P 和 P-D 气泡；小分块在重载下导致 GPU 利用率过低并恶化 TTFT [页码 : 2, 3, 5, 6] [实验支持]	动态分块预填充 (DCP) 机制 [页码 : 6, 7]	图 4 比较了 128 和 2048 静态分块在不同请求率下的气泡延迟 [页码 :6]；图 9、10 证明 DCP 取得了明显更好的 Goodput 和更低的尾部延迟 [页码 : 10, 11] [实验支持]	混合 Prompt 长度极度倾斜（如绝大部分为超长 context）时的自适应调节能力 [页码 : 10] [基于文中逻辑的推断]
Niyama	根据单个节点的 SLO 状态动态调节预填充 Token 数 [页码 : 15] [背景知识]	仅基于单节点局部状态，忽略流水线并行的结构性约束（如流水线深度和各阶段协调），在流水线并行下仍无法消除气泡 [页码 : 15] [作者明确陈述]	将流水线深度 D_p 纳入决策边界的动态分块调度算法 [页码 : 7, 8]	测试了不同 SLO 紧缚度下的 Goodput 表现（图 12） [页码 : 12, 13] [实验支持]，但未提供与 Niyama 系统的直接端到端量化对比 [页码 :10, 15] [实验支持]	与 Niyama 本身的直接端到端性能与计算开销对比 [页码 : 15] [基于文中逻辑的推断]
Splitwise, Mooncake, Dist-Serve	物理分离 prefill 与 decode 以消除计算干扰 [页码 :15] [背景知识]	模型参数双重存储，节点间实时搬运庞大 KV 缓存需要极高网络通信带宽，引入巨大的系统性开销 [页码 : 15] [作者明确陈述]	在同一 GPU 节点上通过 DCP 和 DS 统一调度两阶段 [页码 : 15]	论文未提供直接端到端量化对比，但展示了 ALP 自适应预测算法单步耗时极低 (0.0028 ms) 且存储小 (0.643KB) [页码 : 14, 203 (表格)] [实验支持]	在网络带宽充足或 KV Cache 传输优化技术引入时，DCP 是否仍优于相位解耦方案 [页码 : 15] [基于文中逻辑的推断]

[基于文中逻辑的推断] 为了具体评估本研究各核心主张的证据支撑强度，以下以表格形式展示了主张地图与证据计划，总结了每个主张所对应的实验、直接证据、证据强度以及尚未被完全覆盖的验证条件。

主张与证据强度

主张	对应实验/表图	直接证据	证据强度	尚未覆盖的条件
----	---------	------	------	---------

主张 1：在仅有 PCIe 4.0 互连的 GPU 服务器上，流水线并行 (PP) 经过合理配置可反超张量并行 (TP) 的好吞吐量 (Goodput) [页码 :3, 11] [实验支持]。	图 1 [页码：4]、图 9 和图 10 [页码：10, 11]	4 卡 PCIe A100 上，Qwen2.5-32B 运行于 AzureConv，静态 PP 的 Goodput 在 SLO 限制下 (3.25 req/s) 略逊于 TP (4.0 req/s)，但在引入 DCP 调度后大幅反超 TP [页码：3, 11] [实验支持]。	强（在所测硬件与工作负载下有完整的数据对比）。	跨节点多机分布式网络（如低速以太网）、其他类型的 LLM 架构、以及 8 卡以上更大规模的 PP [页码 :9] [基于文中逻辑的推断]。
主张 2 动态分块 (DCP) 能有效减少预填充主导负载中的 P-P 和 P-D 气泡,在降低 TPOT 与 E2E 延迟的同时基本维持 Goodput [页码：11] [实验支持]。	图 9 和图 10 [页码：10, 11]	对比 PP_{static} ，在 AzureConv 下，DCP_pred 将 32B 模型的 TPOT/E2E 延迟降低 35%/31%，将 14B 模型的 TPOT 降低 50%；在 CNN 轨迹下将 32B 模型的 TPOT/E2E 降低 42%/36%，且 Goodput 得到改善或基本不变 [页码：11] [实验支持]。	强（涵盖了多个真实工作负载和不同参数规模的模型）。	极度倾斜或非泊松分布的请求流模型，以及非 A100 的异构硬件设备 [页码：9, 10] [基于文中逻辑的推断]。
主张 3：自适应延迟预测 (DCP_pred) 比贪婪调整 (DCP_greedy) 能更稳定地减少尾部延迟 (P99/P90) [页码 :11] [实验支持]。	图 10 [页码：11]	在 AzureConv (32B) 中，DCP_pred 降低 TPOT/E2E 延迟为 35%/31%，优于 DCP_greedy 的 19%/18%；在 14B 模型中其 TPOT 降低 50%，亦优于 Greedy 的 40% [页码：11] [实验支持]。	中等（缺少在更剧烈或瞬时爆发流量下的鲁棒性对比）。	流量瞬时大爆发 (traffic burst) 下的响应及收敛时间 [页码 :7, 8] [基于文中逻辑的推断]。
主张 4：延迟调度 (DS) 能平抑生成主导负载中的 D-D 不平衡气泡，提升 Goodput 且缩短 E2E 延迟，代价是少量被抢占请求的 P99 TPOT 恶化 [页码 :11, 13] [实验支持]。	图 10 和图 11 [页码：11, 13]	在 CNN-Reversed（生成重负载）下，DCP_pred+DS 使 Goodput 较 PP 基线提升多达 1.4 倍，平均 E2E 延迟缩短 4062ms，P99 缩短 8088ms；但 P99 TPOT 恶化了 15% [页码：13] [实验支持]。	强（给出了正反两方面的量化数据）。	不同的抢占阈值 (threshold) 配置曲线及其在不同 batch size 下的敏感度分析 [页码 :9] [基于文中逻辑的推断]。
主张 5：自适应延迟预测模型 (ALP) 能够以极其轻微的计算和存储开销实现在线迭代延迟的高精度建模，且无需离线预训练 [页码：8, 14, 15] [实验支持]。	页码 14 和 15 的分析以及页码 203 表格	结合 Prefix/Prefill/Dec Attention 与残差修正后，ALP 预测误差 MAPE 降至 3.26%，单步推理仅耗时 0.0028ms，存储大小 0.643KB，无需离线预训练 [页码：14, 203 (表格)] [实验支持]。	强（有详尽的消融数据和与回归、决策树、DNN 的精确对比）。	更加复杂的模型架构（如混合专家模型 MoE）或具有动态稀疏注意力的模型 [页码 :8] [基于文中逻辑的推断]。

9. 图表解读与论证关系

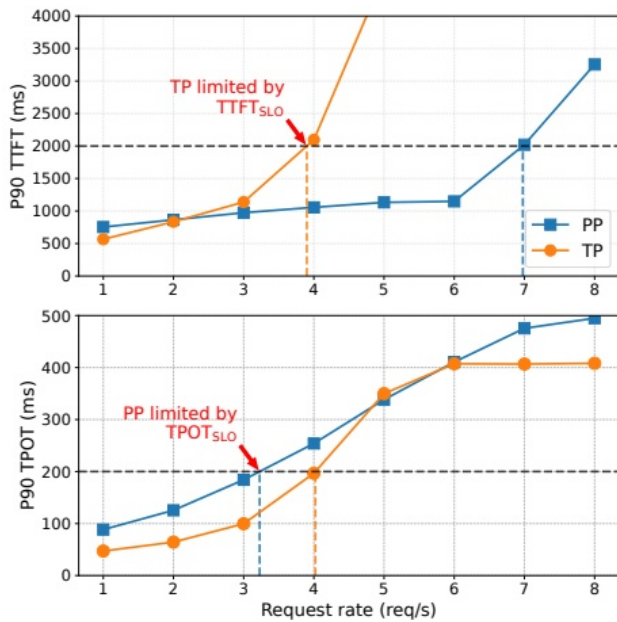


Figure 1: Performance comparison for pipeline and tensor parallelism in serving Qwen2.5-32B with the Azure Conversation workload on 4 NVIDIA A100 GPUs

来源：OCR 第 4 页。

图像解读 验证问题：在缺乏高带宽 NVLink 互连的 PCIe 环境下，对比流水线并行 (PP) 与张量并行 (TP) 在服务大语言模型 (LLM) 时的端到端性能表现，指出 PP 虽在 Prefill 阶段有较低的首字延迟 (TTFT)，但由于流水线气泡导致其单字生成延迟 (TPOT) 较差，从而确立解决流水线气泡的必要性。具体验证不同请求速率下 PP 和 TP 的 P90 TTFT 与 P90 TPOT 变化趋势，以及在给定 TTFT SLO (2000 ms) 和 TPOT SLO (200 ms) 限制下，两者的最大好吞吐量极限。

直接结论：随着请求速率上升，TP 的 P90 TTFT 相比 PP 上升得更快，在 4 req/s 时正好突破 2000 ms 的 SLO 限制；而 PP 的 P90 TTFT 增长缓慢，直到 7 req/s 才突破 2000 ms 的 SLO。但在下图中，PP 的 P90 TPOT 在所有请求速率下都高于或等于 TP，且 PP 在约 3.25 req/s 时就突破了 200 ms 的 SLO；而 TP 在 4 req/s 时达到约 200 ms 的 SLO。因此，TP 受到 TTFT SLO 限制，最大好吞吐量限制在 4.0 req/s；PP 受到 TPOT SLO 限制，最大好吞吐量限制在约 3.25 req/s。

设计含义：该图作为必要性证据，指明了只采用静态调度的 PP 在在线服务时的气泡瓶颈，引导了后续论文设计动态调整分块大小 (DCP) 以及延迟调度 (DS) 来优化 TPOT 并提高好吞吐量的设计动机。该图本身并不能直接证明后续提出的 DCP 和 DS 机制在实际减少气泡和提高吞吐方面的有效性。

如何阅读：横轴代表请求速率 Request rate (req/s)，取值范围 1 到 8；上图纵轴代表 P90 TTFT (ms)，取值范围 0 到 4000；下图纵轴代表 P90 TPOT (ms)，取值范围 0 到 500。右侧图例中，蓝色方形代表 PP，橙色圆形代表 TP。黑色水平虚线表示对应的 SLO 阈值。阅读时，首先看上图比较不同请求速率下 PP 与 TP 的 P90 TTFT 差异，确认 TP 在何处 (4 req/s) 突破 2000 ms SLO 虚线；然后看下图比较两者的 P90 TPOT 差异，确认 PP 在何处 (约 3.25 req/s) 突破 200 ms SLO 虚线，从而评估两者在 SLO 约束下的最大支持吞吐量。

适用范围与证据缺口：本实验基于 4 张 NVIDIA A100 (PCIe 4.0) GPU 服务 Qwen2.5-32B 模型，输入使用 Azure Conversation 工作负载，SLO 阈值设为 P90 TTFT = 2000 ms，P90 TPOT = 200 ms。无额外证据缺口。

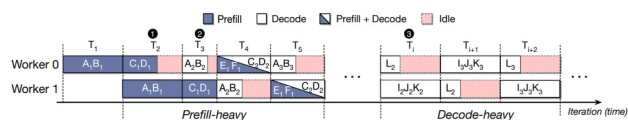


Figure 2: Three types of pipeline bubbles (The subscript indicates the iteration number.)

来源：OCR 第 5 页。

图像解读 验证问题 形象解释在线 LLM 服务在使用流水线并行时,由于静态微批次调度和工作负载动态波动,在时序上如何产生 Prefill-Prefill (P-P) 不平衡、Prefill-Decode (P-D) 不平衡以及 Decode-Decode (D-D) 不平衡这三种负载不平衡,并由此引发空闲等待时间(气泡)。**直接结论**:即使在同一次迭代中,Worker 0 和 Worker 1 上的任务执行时间也不相同,从而在较早完成任务的 Worker 上交替产生粉色斜线的 Idle 气泡。具体地,在 Prefill-heavy 场景下,在 T_2 步由于 microbatch 间 prefill 输入序列长度差异引发了 P-P 不平衡气泡(①);在 T_3 和 T_4 步由于 prefill 任务和 decode 任务混合在微批次中执行引发了 P-D 不平衡气泡(②)。在 Decode-heavy 场景下,从 T_i 到 T_{i+2} ,由于各微批次中 decode 请求数量不均引发了 D-D 不平衡气泡(③)。

设计含义:指明了针对长 Prompt 导致的 P-P 与 P-D 不平衡,需要设计动态调整分块大小的机制(DCP)来拉平 prefill 耗时;指明了针对 decode 数量不均导致的 D-D 不平衡,需要设计迁移与抢占以对齐请求数的负载平衡机制(DS)。该图为机制示意图,无法用作证明 DCP 和 DS 方法实际降低气泡和提升性能效果的实验证据。

如何阅读:纵轴代表不同的流水线 Worker 阶段(Worker 0 与 Worker 1);横轴代表时间演进的各个迭代步($T_1 \sim T_5$ 和 $T_i \sim T_{i+2}$)。深蓝代表 Prefill 阶段,白色代表 Decode 阶段,斜切蓝白代表混合阶段,粉色斜线条纹代表 Idle。块中的 A_1B_1 等表示微批次名称与迭代序号的组合。沿着时间横轴由左至右阅读,对比上下两行 Worker 块在相同迭代步中的横向宽度。若某一步中一个 Worker 的块较窄,其后面紧跟的粉色斜线条纹区域即代表其产生的空闲气泡,对比两 Worker 的计算不平衡度。

适用范围与证据缺口:本机制图基于一个简化的两阶段流水线(2 stages, Worker 0 & Worker 1),假设最大 batch size 为 3。无额外证据缺口。

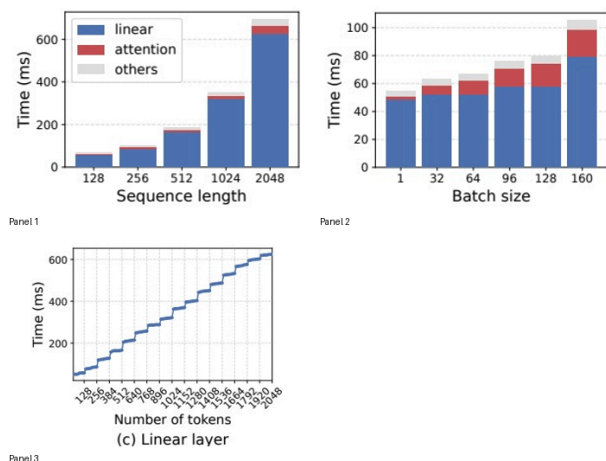


Figure 3: Performance characteristics: (a) prefill time when varying the sequence length, (b) decode time when increasing the batch size, and (c) linear operations when increasing the number of tokens
来源：OCR 第 5 页。

图像解读 验证问题：量化分析预填充 (prefill) 和生成 (decode) 阶段由于序列长度和批次大小变化对计算延迟的影响，进而揭示流水线不平衡的底层硬件执行特性，作为提出动态调整分块大小 (DCP) 和延迟调度 (DS) 技术的微观机制和必要性依据。具体验证：一是预填充延迟主要取决于输入序列长度；二是生成阶段延迟与批次大小并非平滑的线性关系，而是由于硬件计算内核 (kernel) 的对齐特性而呈现出阶梯函数的效率变化趋势，解释 Decode-Decode (D-D) 不平衡的硬件效率根源。

直接结论：在 Panel 1 (a) 中，随着 Sequence length 的成倍增加，预填充阶段的总耗时显著上升，其中 linear 算子 (蓝色) 在所有长度中均占据了绝大部分的执行时间且呈线性增长，而 attention 算子 (红色) 占比非常微小。在 Panel 2 (b) 中，随着 Batch size 从 1 增加到 160，生成阶段的单步时间呈阶梯式增长而非线性上升；当 batch size 从 128 增长到 160 时，耗时出现显著的阶梯性跳跃 (上升约 31%)，且 linear 依旧是时间消耗的主导者。在 Panel 3 (c) 中，线性层算子的计算耗时表现出极强的阶梯状特征，每当 token 数量超过 128 的倍数时，延迟都会发生一次突然的阶梯式向上跃迁，而在两个倍数区间内部，耗时基本保持水平恒定。

设计含义：预填充阶段耗时高度依赖序列长度，验证了在流水线并行下，不同微批次中 prompt 长度的剧烈波动会直接导致巨大的 prefill-prefill (P-P) 负载失衡，说明了采用分块预填充的必要性。生成阶段线性算子耗时的阶梯效率特性，直接支持了延迟调度 (DS) 中基于阈值的抢占设计，即如果微批次大小略微超出硬件友好边界 (以 128 为倍数)，通过暂时抢占多余的少量请求，使其批次大小向下对齐至对齐边界 (例如从 129 降到 128)，从而规避阶梯上升带来的 31% 额外延迟，重平衡 Decode-Decode 气泡，使整个流水线维持在更高效的算力状态。

如何阅读：由三幅子图组成。Panel (a) 和 (b) 位于上方左右并排，为堆叠柱状图，横坐标分别为 Sequence length 和 Batch size，纵坐标均为 Time (ms)；堆叠柱中蓝色代表 linear 算子，红色代表 attention 算子，灰色代表 others。Panel (c) 位于下方，是二维折线/阶梯图，横轴为 Number of tokens (并标有 128 的倍数刻度)，纵轴为 Time (ms)，表示线性层耗时。阅读时，比较在不同 token 数 (尤其是跨越 128 的倍数对齐边界时) 线性层耗时的阶梯状跳变，以及堆叠柱中不同部分所占的时间比例。

适用范围与证据缺口：本证据覆盖了 Qwen2.5-32B 模型在单节点四卡 PCIe 4.0 NVIDIA A100 GPU 服务器上的算子耗时表征。无额外证据缺口。

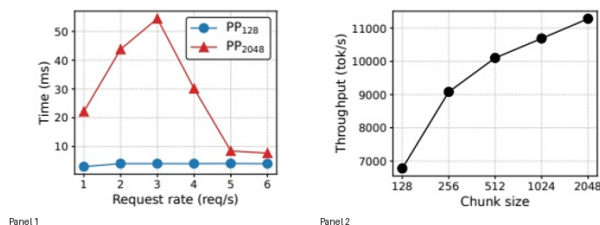


Figure 4: Pipeline bubbles and prefill token throughput according to load and chunk size (when serving Qwen2.5-32B with the Azure Code workload on 4 NVIDIA A100 GPUs)

来源：OCR 第 6 页。

图像解读 验证问题：分析静态分块预填充（static chunked-prefill）面临的“气泡时间”与“吞吐量”两难困境（Trade-off），从而说明在流水线并行中，固定大小的分块无法同时适应多变的请求率（负载）和保持高 GPU 利用率，论证引入“动态分块调整（DCP）”的核心必要性。具体验证：第一，小分块大小（如 128）在低请求率下能将流水线气泡降到极低，但会使总体的 Token 吞吐量受到严重折损；第二，大分块大小（如 2048）在低中负载下会引发高额的流水线气泡，但在高负载（≥ 4 req/s）下，气泡占比会因为流水线被充分填充而迅速衰减，且大分块提供了明显更高的 token 计算吞吐。

直接结论：在 Panel (a) 中， PP_{128} 的气泡时间在所有请求率下都保持在极低的水平（约 3-4ms）；而 PP_{2048} 在低请求率下气泡时间极高（3 req/s 时达到顶点 54ms），但当请求率达到 4 req/s 时，其气泡时间暴跌至 30ms，并在 5-6 req/s 时降到与 PP_{128} 相当的低位（8ms 左右）。在 Panel (b) 中，随着 Chunk size 从 128 逐渐增大到 2048，Token 吞吐量单调递增（从约 6800 tok/s 陡峭上升到约 11300 tok/s）。

设计含义：该图作为必要性证据，验证了静态分块策略的权衡困境，为本文的核心方法“动态分块调整（DCP）”设计提供了直接依据：系统必须根据当前的运行负载（请求率）和 SLO 限制动态调节分块大小，在轻载或 SLO 的 TPOT 紧张时选择更小的 chunk size 以规避气泡，而在重载或 TTFT 告警时选择更大的 chunk size 以换取吞吐，通过在两条曲线间动态滑动来取得 Pareto 最优。

如何阅读：由两幅折线图组成，左右并排。Panel (a) 为折线图，横轴为请求速率 Request rate (req/s)（范围 1 到 6），纵轴为流水线气泡时间 Time (ms)，包含蓝色圆圈折线（ PP_{128} ）和红色三角形折线（ PP_{2048} ）。Panel (b) 为折线图，横轴为分块大小 Chunk size，纵轴为 Token 吞吐量 Throughput (tok/s)。阅读时应在 Panel (a) 中对比不同请求率下 128 和 2048 两种静态分块大小的气泡延迟差异，并在 Panel (b) 中观察随着分块大小递增吞吐量的变化趋势。

适用范围与证据缺口：本证据覆盖了在 4 卡 PCIe 4.0 NVIDIA A100 GPU 平台上，使用 Azure Code 工作负载服务 Qwen2.5-32B 模型时的流水线气泡时间和预填充吞吐表现。无额外证据缺口。

[基于文中逻辑的推断] 在系统限制方面，除上述 4 张图外，其余 14 张图由于未进行视觉分析，无法从当前内容中确认其视觉设计、直接传达的信息和支撑论点。

10. 完整因果推理树

- [基于文中逻辑的推断] 根问题：在单卡 GPU 显存不足的大模型服务中，如何在低带宽 PCIe 互连下，在保障首字延迟（TTFT）和单字生成延迟（TPOT）SLO 的前提下最大化系统的好吞吐量（Goodput）[页码：2, 3]
- [实验支持] 旧方法限制：张量并行（TP）在 PCIe 下通信开销极大限制了多卡加速[页码：2, 4]；静态流水线并行（PP）由于流量波动和提示词长度不均产生严重的 P-P、P-D 和 D-D 不平衡气泡[页码：3, 5]；静态分块预填充面临小分块吞吐低与大分块低载下气泡大的两难困境[页码：2, 3]
- [基于文中逻辑的推断] 核心洞见/假设：根据系统负载与 SLO 约束动态调整预填充分块大小（DCP）以折中气泡与吞吐[页码：6, 7]；利用自适应延迟预测模型（ALP+RLS）主动评估延迟，规避贪婪调整的反馈滞后[页码：7, 8]；延迟调度（DS）在生成阶段通过微批次迁移与硬件友好边界对齐抢占消除 D-D 气泡[页码：9]
- [基于文中逻辑的推断] 方法模块 1：动态分块调度 DCP (Greedy 与 Predictive) [页码：7, 8]
 - [基于文中逻辑的推断] 可验证预测 1A：DCP 相比静态 PP 在预填充主导负载下能有效降低 TPOT/E2E 延迟并提升 Goodput [页码：10, 11]
 - [实验支持] 指标与实验：在泊松分布请求下评估 AzureConv, CNN, ShareGPT 轨迹的 Goodput 和延迟分布[页码：10]
 - [实验支持] 图表证据：图 9、图 10 中 DCP_pred 相比 PP_{static} 的 TPOT/E2E 延迟降低和好吞吐量对比[页码：11]
 - [实验支持] 结论与适用边界：在 4 卡 PCIe A100、Qwen2.5 32B/14B 模型上，DCP_pred 分别使 AzureConv 轨迹的 TPOT 降低 35% 和 50%，有效消除了预填充阶段的气泡，但在跨节点或更大流水线深度下的泛化效果未验证[页码：11]
 - [基于文中逻辑的推断] 可验证预测 1B：Predictive DCP 比 Greedy DCP 在应对剧烈变化的负载时有更稳定的尾部延迟[页码：7, 8]
 - [实验支持] 指标与实验：评估并对比 DCP_pred 与 DCP_greedy 在不同轨迹下的 P90/P99 延迟表现[页码：11]
 - [实验支持] 图表证据：图 10 中 DCP_pred 在 AzureConv (32B) 中 TPOT/E2E 降低 35%/31%，优于 DCP_greedy 的 19%/18% [页码：11]

- [实验支持] 结论与适用边界：自适应预测调度通过克服反馈滞后，在减少尾部延迟上较贪婪调度具有更明显且稳定的优势 [页码：11]
- [基于文中逻辑的推断] 可验证预测 1C：自适应延迟预测模型（ALP）能低延迟高精度地在线预测 LLM 迭代耗时 [页码：8]
 - [实验支持] 指标与实验：测量 ALP 模型在不同组件消融下的平均绝对百分比误差（MAPE）、计算延迟和模型大小，并与 ML 模型对比 [页码：14, 15]
 - [实验支持] 图表证据：页码 203 表格中 ALP 结合残差修正后 MAPE 降至 3.26%，单步推理仅 0.0028ms [页码：14, 203]
 - [实验支持] 结论与适用边界：ALP 无需繁琐重训即可提供支持分块决策的高精度在线延迟预测，但尚未验证在 MoE（混合专家模型，将大模型内部前馈网络拆分成多个独立专家子网络并在运行时按需路由激活的架构模型 [页码：8]）等更复杂架构模型上的效果 [页码：15]
- [基于文中逻辑的推断] 方法模块 2：延迟调度 DS（微批次对齐抢占与迁移重构）[页码：9]
 - [基于文中逻辑的推断] 可验证预测 2A：在生成主导的负载中，延迟调度能通过微批次重组对齐硬件高效边界，显著平抑 D-D 气泡并提升吞吐 [页码：9]
 - [实验支持] 指标与实验：在 CNN-Reversed 轨迹下评估 DCP_pred+DS 的 E2E 延迟、P99 TPOT 和 Goodput [页码：11, 13]
 - [实验支持] 图表证据：图 10 和 图 11，显示 DCP_pred+DS 使 Goodput 较 PP 基线提升达 1.4 倍，E2E 延迟缩短，但 P99 TPOT 恶化 15% [页码：13]
 - [实验支持] 结论与适用边界：延迟调度对生成主导的负载有显著优化，但以被抢占请求的 TPOT 性能微幅受损为代价；且当 SLO scale 收紧至 0.6 以下时，其重平衡效果会显著下降 [页码：13]

11. 结论、贡献与边界

- **作者主张的核心贡献:** [作者明确陈述] 为了对本研究的学术定位进行清晰说明，本段总结了作者明确陈述的核心贡献体系，论文主要归纳了在线 LLM 服务流水线并行下的三类气泡模式，提出了包含 Greedy 和 Predictive 两种分块调节调度器的动态分块预填充机制，以及通过微批次迁移与阈值抢占重平衡的延迟调度技术 [页码：5, 6, 7, 8, 9]。
- **证据充分的结论:** [实验支持] 为了展示本研究已被实验完全验证的成果，本段总结了在 4 卡 A100 PCIe 环境下的主干结论，通过引入动态分块调度或合理配置静态分块，PP 的好吞吐量能实现对张量并行的反超；在预填充主导负载中 DCP_pred 可降低 TPOT 延迟达 35%-50%，且对系统无负面影响；在生成主导负载中 DCP_pred+DS 虽使好吞吐量提升多达 1.4 倍，但会造成 P99 TPOT 恶化 15% [页码：3, 11, 13]。[实验支持] 为了展示延迟预测模型的效果，本段总结了自适应预测模型 ALP 的实测表现，在结合残差修正后 ALP 模型预测误差 MAPE 降至 3.26%，单步推理时间极短，相比 DNN 虽然在精度上稍弱 0.70 个百分点，但无需额外的离线预训练开销，展现了极佳的实用价值 [页码：14, 15]。
- **尚属推断的意义:** [基于文中逻辑的推断] 为了展望本方法的泛化能力，本段归纳了其对于更广泛异构计算环境的技术意义，由于 ALP 预测机制能在线校准注意力模型参数，因此 DCP 与 DS 联合调度系统在跨节点多机分布式网络或 AMD GPU、NPU 等异构计算设备下亦可能自动修正硬件误差并保持类似加速优势，但该意义尚未被当前实验直接验证 [页码：4, 9]。
- **适用条件与局限:** [实验支持] 为了揭示该方案的物理与算法适用局限，本段阐明了导致系统失效的特定边界条件，当 SLO 尺度设为极度严苛的 0.2 时，所有方法的 Goodput 均将退化为零导致调度失效；当 SLO 尺度收紧至 0.6 以下时，在 CNN-Reversed 负载下延迟调度（DS）的重平衡效果会显著下降，因为生成请求还未被充分累积就会直接触发 TPOT 限制 [页码：12, 13]。
- **下一步验证与审稿式风险检查:** [基于文中逻辑的推断] 为了以审稿人视角对本方法的学术与设计漏洞进行检查，本段梳理了研究中存在的潜在风险点，首先，延迟调度中对齐阈值具体配置细节的缺失可能会妨碍第三方的高效复现；其次，实验在 4 卡单节点上的局限使得多机分布式跨节点下的扩展性能及 ALP 的累积预测误差仍未被验证；最后，本研究缺少极端脉冲流量（bursty workload）和更复杂模型（如 MoE）下的鲁棒性分析，RLS 算法在极端并发波动下的收敛稳定性仍有待进一步验证 [页码：9, 10, 15]。

12. 面向初学者的学习路径

1. [背景知识] **模型并行（Model Parallelism）**：因为超大语言模型的参数量过于庞大，单个 GPU 显存无法装下，必须将模型进行切分并分布在多张显卡上协同运行，这是理解多卡分布式大模型推理服务的核心基础 [页码：3]。
2. [背景知识] **张量并行（TP）与流水线并行（PP）**：这是模型并行的两种主流切分策略：TP 倾向于将大模型每一层的计算权重切分给多个 GPU 共同横向计算，但在计算过程中需要高频 of collective 同步通信；PP 倾向于纵向装配，将大模型的不同层依次分配给不同的 GPU 阶段，像装配线一样顺序处理微批次，其通信开销极小，但在在线随机流量下极易引发流水线气泡，这是本文立论和改进的两个核心对照路线 [页码：4]。
3. [背景知识] **首字延迟（TTFT）与单字生成延迟（TPOT）**：这是大语言模型推理的两个独特执行阶段：处理输入提示词以产生第一个字符的预填充阶段延迟为 TTFT，而后续逐个字符输出的生成阶段单步平均间隔时间为 TPOT。两者的达标率共同决定了在线大模型服务的高吞吐量与 SLO 保障水平 [页码：3]。
4. [背景知识] **流水线气泡（Pipeline Bubble）**：在流水线并行装配线上，由于各个 GPU 阶段的加工时间不一致，导致某些 GPU 闲置等待上一个阶段完成输入，从而在这段等待时间里处于闲置空载状态的算力浪费，这是本文论文力图消除的底层瓶颈 [页码：2, 4]。
5. [背景知识] **分块预填充（Chunked-Prefill）**：为平抑长输入提示词对生成阶段产生的延迟干扰，而选择将长输入切分成若干个固定大小的小分块片段进行分批计算的技术，它是理解本文如何进行“动态自适应”调整分块大小的演进起点 [页码：2, 3]。